香 港 大 學
THE UNIVERSITY OF HONG KONG



# Autoresearch and RSI

## Can an agent improve how it works?



COMP7607A · LECTURE 7

# Autoresearch: motivation, experiment, evidence

| | | |
|---|---|---|
| 01 | **Overview** | Model training recap and the motivation |
| 02 | **Autoresearch** | Follow one complete experiment |
| 03 | **Evaluation** | Check whether the gain is real |
| 04 | **RSI** | A brief look at improving the researcher |

# Lecture 2: a small GPT learns from text



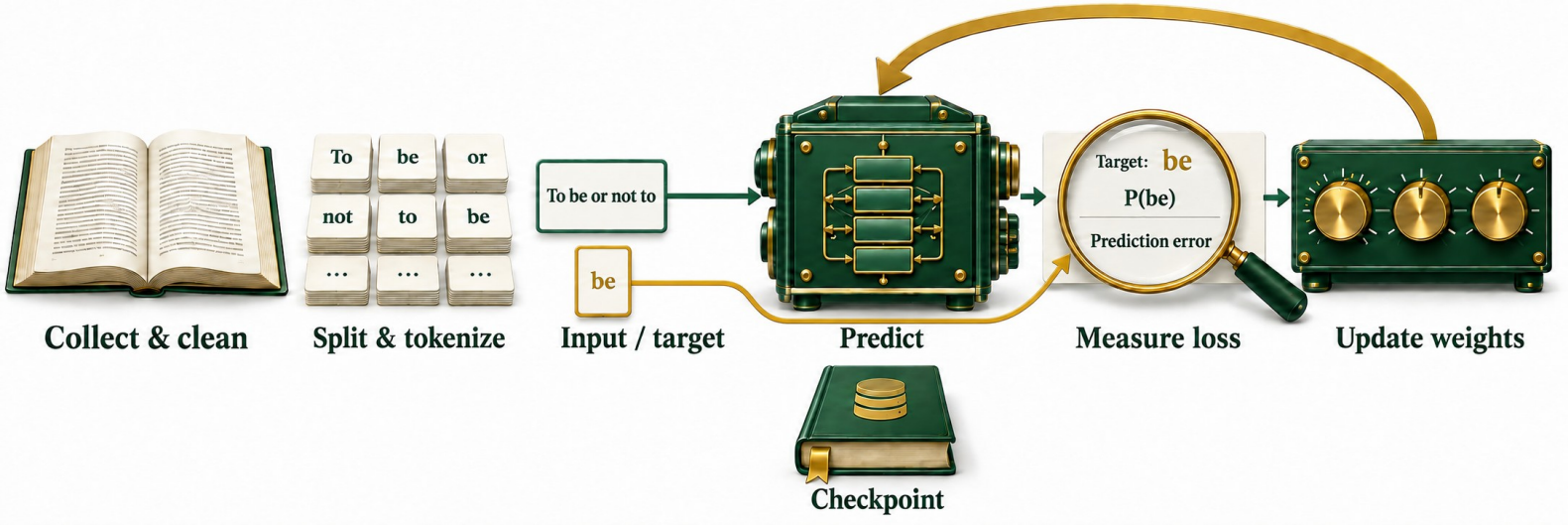
## Tiny Shakespeare

## To be, or not to

Next **be**

Source: Lecture 2, NanoGPT character recipe. Portrait: Wikimedia Commons.

# The learning cycle from Lecture 2

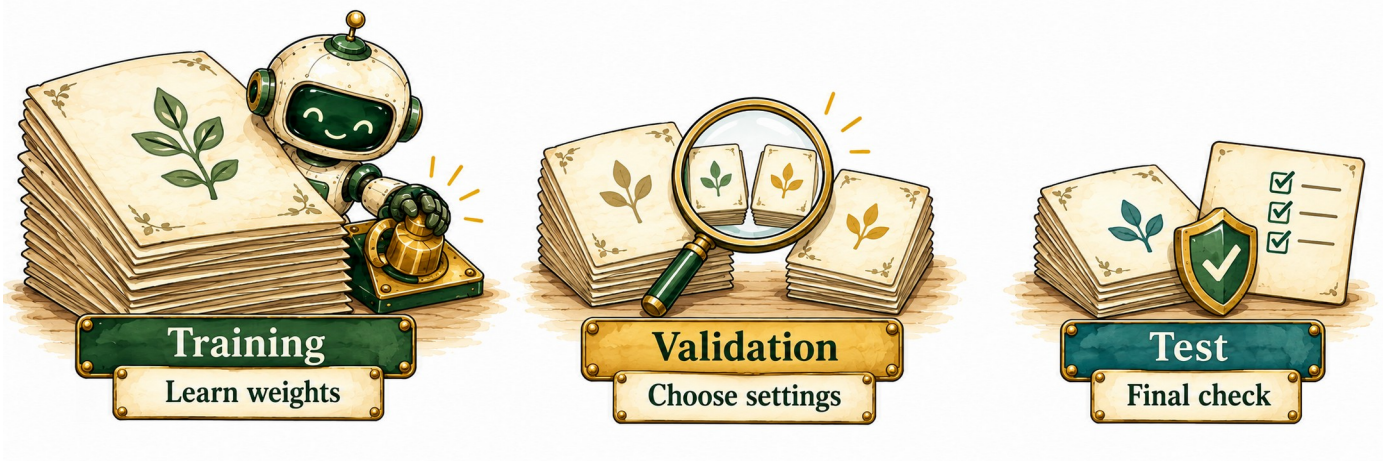



**Text supplies the targets. The optimizer updates the weights.**

Source: Lecture 2, training framework illustration.

# Training, validation and test data

## Split documents first. Build examples within each split.




## Only training batches update the model weights.

Source: Lecture 2. General evaluation practice, not three built-in autoresearch splits.

# Batch: one group. Epoch: all examples once

**Batch 1**
Examples 1–6

**Batch 2**
Examples 7–12

**Batch 3**
Examples 13–18

**Batch 4**
Examples 19–24

## Inside Batch 1: six text examples

| Example | Input tokens | Next-token targets |
|---|---|---|
| 1 | To be or not | be or not to |
| 2 | be or not to | or not to be |
| 3 | the cat sat on | cat sat on the |
| 4 | we read a book | read a book today |
| 5 | she opened the door | opened the door slowly |
| 6 | a bird can fly | bird can fly high |

**24 examples = 4 batches = 1 epoch**

Teaching examples. Words stand in for tokens. Targets shift one position. No gradient accumulation here.

# What if there are 26 examples?

## Batch size = 6

| Batch 1 | Batch 2 | Batch 3 | Batch 4 | Batch 5 |
|---|---|---|---|---|
| 6 | 6 | 6 | 6 | 2 |

## Keep the last 2

`drop_last=False`

5 batches use all 26 examples

## Or drop the last 2

`drop_last=True`

4 full batches use 24 examples

Source: PyTorch DataLoader, drop_last. Teaching example: batch size 6, no gradient accumulation.

# Why train in batches?

**A batch lets the GPU work on several examples together.**

| | |
|---|---|
| **1 example** | Little parallel work. A noisy gradient estimate. |
| **6 examples** | More parallel work. Average their training signal. |
| **All 24 examples** | Works for this toy set. A real dataset may not fit. |

**Choose a batch that fits memory and trains efficiently.**

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

# How one batch updates the weights

**Here, we update after every batch of 6 examples.**

## 1 forward

Predict tokens
and average the loss

## 1 backward

Calculate gradients
from the batch loss

## 1 update

optimizer.step()
changes the weights

**24 examples: 4 batches = 4 forwards + 4 backwards + 4 updates**

**26 examples: keep the last 2 = 5 of each per epoch**

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

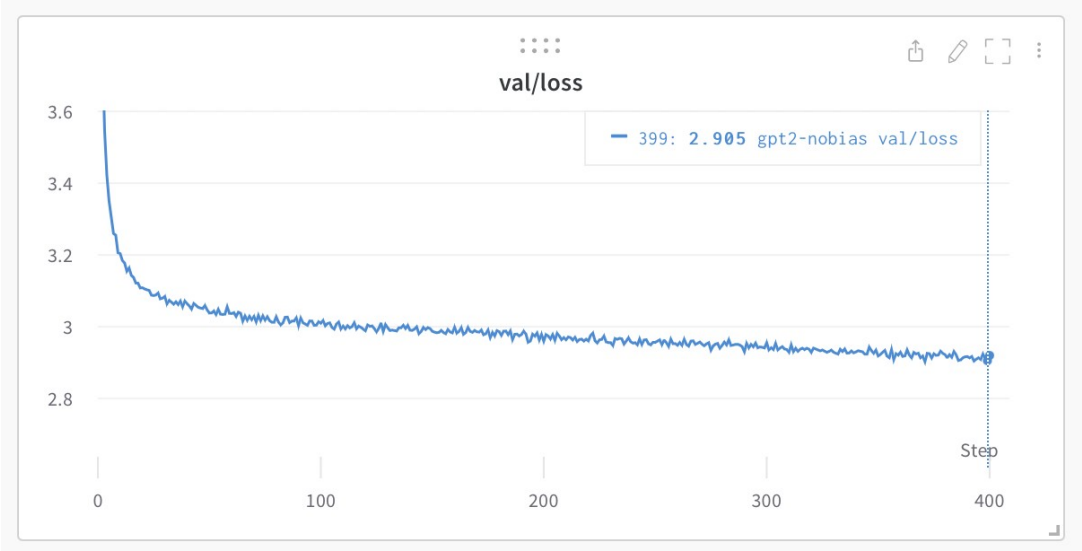
# The training loop updates the weights

```
for epoch in training_passes:                    # Read the training examples again
    for x, y in training_batches:                # x: token rows. y: shifted targets
        scores = model(x)                        # Predict the next token at each position
        loss = compare(scores, y)                # Compare predictions with true tokens
        update_weights(loss)                     # Update the small GPT after this batch
```

**Validation checks predictions without updating weights.**

Source: Lecture 2 / NanoGPT. Teaching pseudocode. No gradient accumulation in this example.

# Lecture 2: a real NanoGPT learning curve




## One training run

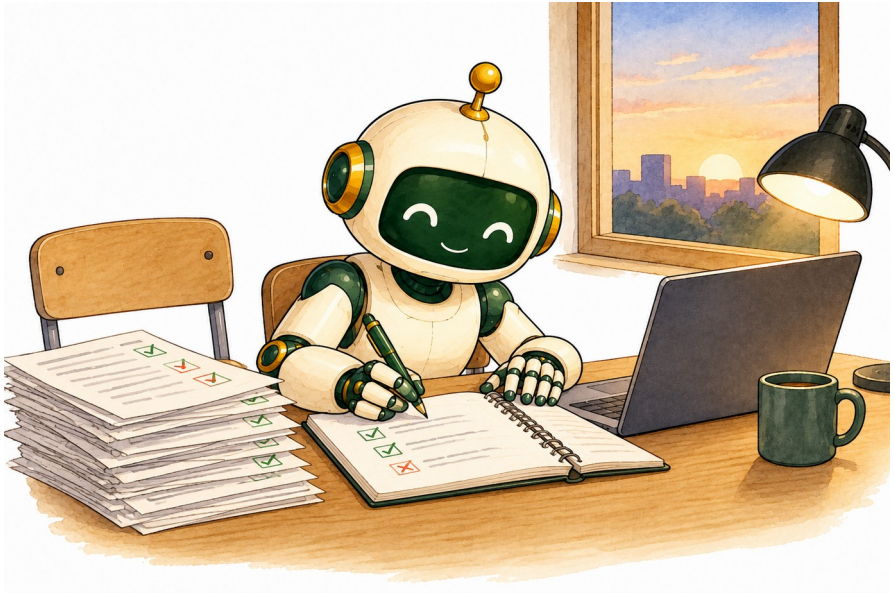
New text checks what it learned.

## Who chooses the next experiment after this run?

Source: Karpathy nanoGPT, GPT-2 (124M), OpenWebText. Reused from Lecture 2.

# Could it try the next experiment while we sleep?

**Change one idea.**
**Run a test.**
**Read the result.**



**We still need a clear goal and a reliable check.**

Source: Karpathy autoresearch; conceptual course illustration

# Why built Autoresearch

**Andrej Karpathy @karpathy**

**“The goal is to engineer your agents to make the fastest research progress indefinitely and without any of your own involvement.”**

The agent proposes and tests changes, then uses the results.

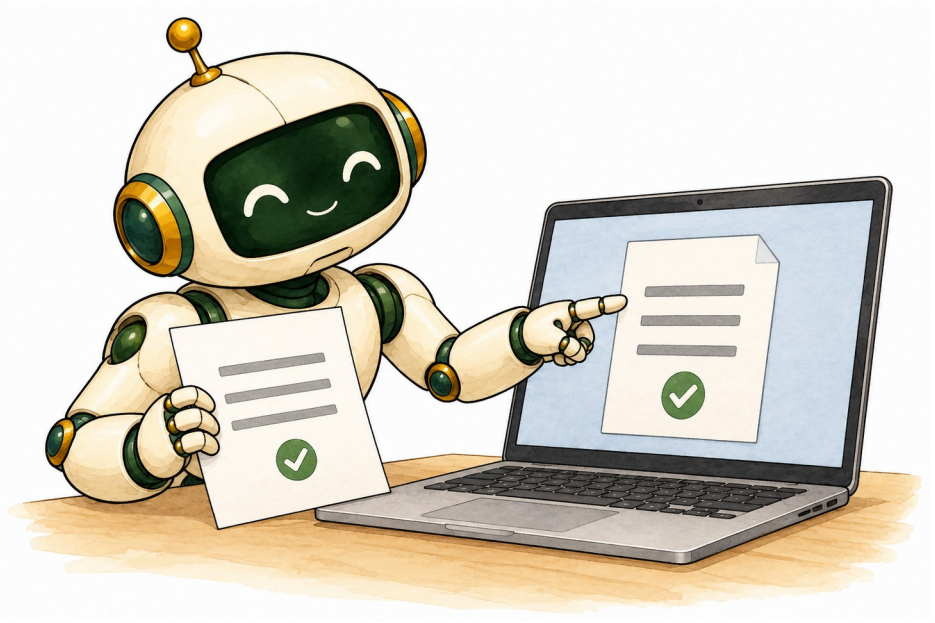
**Small-model training is one testbed for this broader idea.**

Source: @karpathy, 7 March 2026 (UTC).

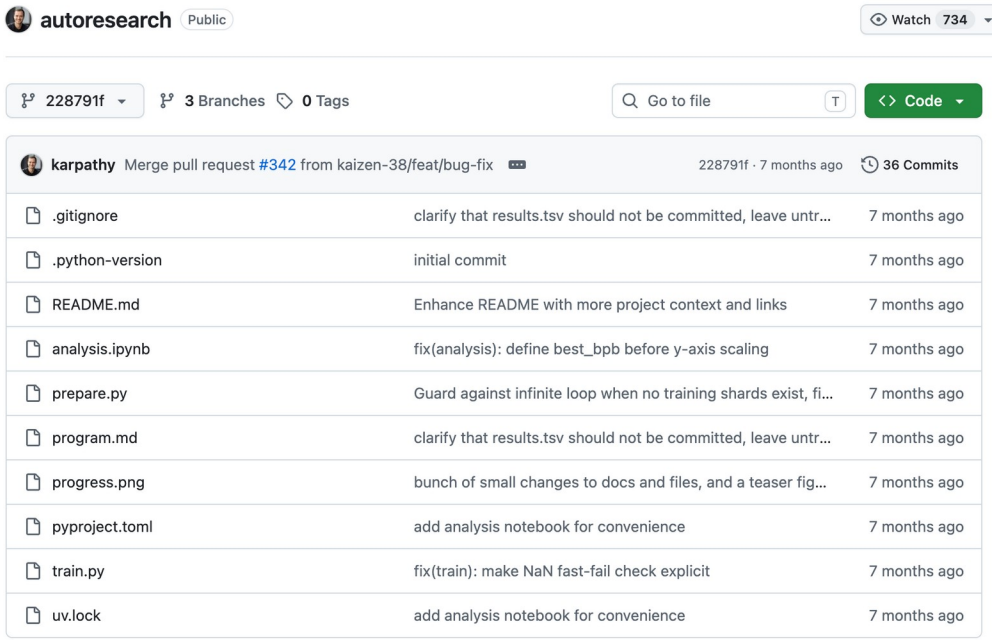
02

# Autoresearch

A coding agent runs a small research project.



# Inside the Autoresearch repository




Source: github.com/karpathy/autoresearch, snapshot 228791f. GitHub screenshot.

| | |
|---|---|
| **train.py** | Model + training |
| **prepare.py** | Data + evaluation |
| **program.md** | Agent instructions |
| analysis.ipynb | Analyze results |
| README.md | Project guide |
| pyproject.toml | Dependencies |
| uv.lock | Locked versions |
| .python-version | Python version |
| .gitignore | Ignored files |
| progress.png | Example plot |

# NanoGPT, nanochat and Autoresearch

NanoGPT

**Our recap: how GPT learns from text**

## nanochat

Train and use
a chatbot

**Simplified training for one GPU**



## Autoresearch

An agent edits and tests
the training recipe

**Autoresearch keeps its own train.py and prepare.py.**

Source: Karpathy official READMEs: nanoGPT / nanochat / autoresearch. Code snapshot 228791f.

# How big is nanochat?

## The original 2025 example: --depth=20

**0.56B**

about 560 million parameters

Exact count: 560,988,160

| | |
|---|---|
| 20 | Transformer layers |
| 1,280 | features per token |
| 10 | attention heads per layer |

**Change the depth to train a different-sized model.**

Parameter counts also depend on the architecture and vocabulary.

Source: Karpathy, Introducing nanochat (Oct 2025), training log. nanochat README.

# train.py: settings the agent can change

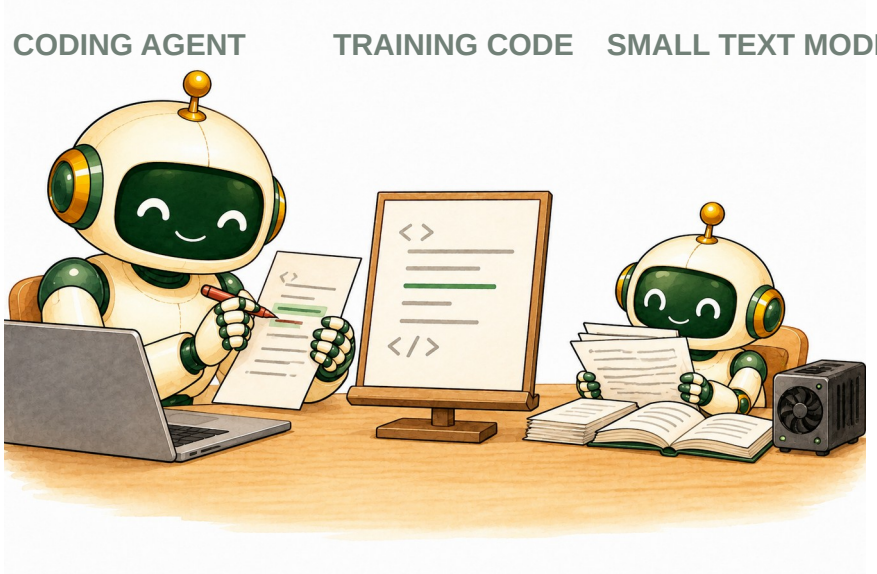
## Autoresearch code: highlighted values can be changed.

```
DEPTH =                 8          # number of Transformer layers
ASPECT_RATIO =          64         # target width / depth
HEAD_DIM =              128        # features in each attention head
TOTAL_BATCH_SIZE =      2**19      # tokens per weight update
DEVICE_BATCH_SIZE =     128        # sequences per forward/backward
MATRIX_LR =             0.04       # learning rate for weight matrices
WEIGHT_DECAY =          0.2        # weight shrinkage strength (Muon)
```

**The agent can also edit the model and optimizer code.**

Source: Karpathy autoresearch train.py, snapshot 228791f. Selected lines, comments added.

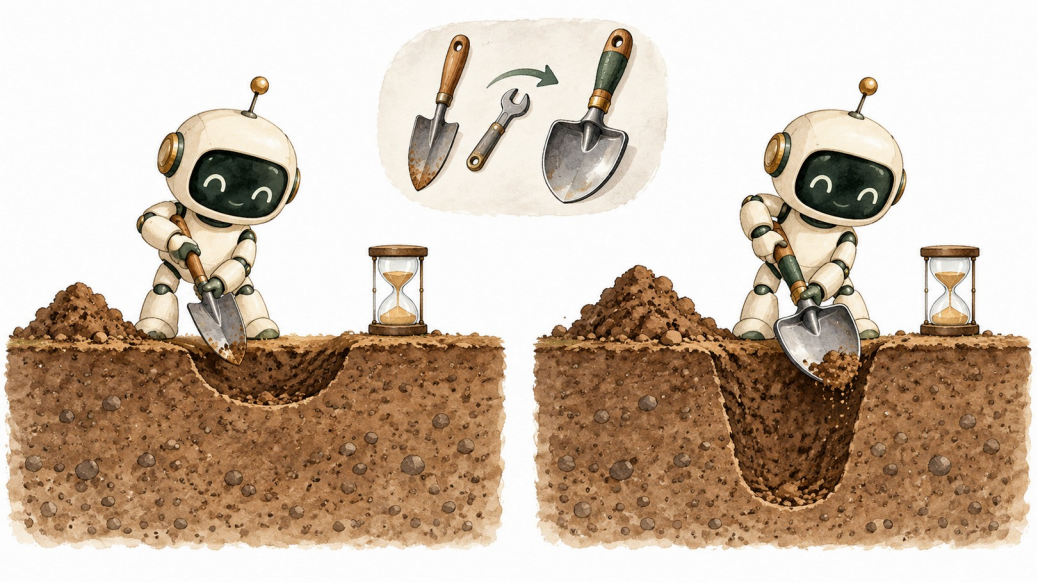
# Small-model training: one Autoresearch example




**The coding agent edits train.py. The small model learns.**

Source: Karpathy autoresearch; conceptual illustration

# Same time. Can a better shovel dig deeper?

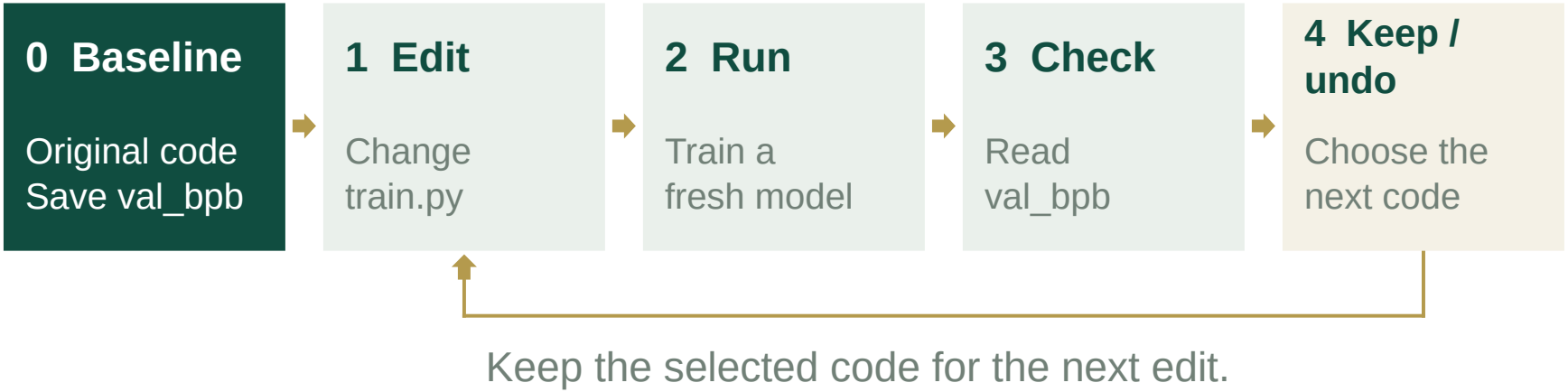


## Change the training method. Keep the training budget fixed.

Course analogy. In the experiment, a lower val_bpb means a better result.

# Autoresearch repeats one experiment loop

## Run once to set the baseline




**Baseline once. Then repeat steps 1–4.**

Source: Karpathy program.md. Simplified overview.

# Three files define the experiment

## `program.md`

**The research instructions**

Human defines the goal

## `train.py`

**The training recipe**

Agent edits this file

## `prepare.py`

**Data and evaluation**

Keep this fixed

**A small editable surface makes comparisons easier.**

Source: Karpathy autoresearch, snapshot 228791f

# train.py: the experiment the agent can change

### The small language model

```
class GPT(nn.Module):
```

### Tokens before each update

```
TOTAL_BATCH_SIZE = 2**19
```

### The weight-update rule

```
class MuonAdamW(torch.optim.Optimizer):
```

**The agent changes this file, then tests the result.**

Source: Karpathy train.py, snapshot 228791f. Original code excerpts, with omissions.

# Where do 8, 64 and 128 come from?

## The author sets these starting values in train.py.

```
DEPTH = 8                    # choose 8 Transformer layers
ASPECT_RATIO = 64            # target width = layers × 64
HEAD_DIM = 128               # choose 128 features per head
```

**Test other values with the same data and five-minute budget.**

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

# Where do training settings come from?

## Hyperparameters are settings we choose.

| | |
|---|---|
| **Data and task** | How much text should the model read at once? |
| **GPU memory** | Room for the model and saved training calculations<br>Larger model, batch or context = more memory needed |
| **Experience + tests** | Try a learning rate. Compare validation results. |

**Out of memory? Reduce batch, context length or model size.**

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

# The code calculates width and head count

**With the chosen settings: 8 layers, ratio 64, head size 128**

MODEL WIDTH

**8 layers × 64 = 512 features per token**

ATTENTION HEADS

**512 features ÷ 128 per head = 4 heads**

**Try 10 layers instead: width 640, with 5 heads.**

The code rounds width up to a multiple of 128 when needed.

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

# train.py: when do the weights change?

## Simplified training code for the default two small batches

```
model.zero_grad()                    # clear old gradients
for x, y in two_batches:             # load 128 sequences each time
    loss = model(x, y)               # forward: predict + compare
    (loss / 2).backward()            # backward: add half the gradient
optimizer.step()                     # update weights ONCE
```

**Validation: forward only. No backward. No weight update.**

Source: Karpathy autoresearch train.py / prepare.py, snapshot 228791f. Classroom explanation.

# program.md tells the agent what a good result means

**program.md**

| | |
|---|---|
| **Goal** | Lower validation bits per byte |
| **Edit** | train.py only |
| **Compare** | Same data and training budget |
| **Record** | Code change, score and outcome |

Source: Karpathy program.md; shortened paraphrase

# val_bpb: validation bits per byte

## Validation

Text held out from training

## Bits per byte

Prediction loss, measured per byte of text

**Total prediction loss in bits ÷ total text bytes**

**Same validation text. Lower is better.**

Source: Karpathy prepare.py, evaluate_bpb()

# program.md: the actual research instructions

## Experimentation

Each experiment runs on a single GPU. The training script runs for a **fixed time budget of 5 minutes** (wall clock training time, excluding startup/compilation). You launch it simply as: `uv run train.py`.

**What you CAN do:**

- Modify `train.py` — this is the only file you edit. Everything is fair game: model architecture, optimizer, hyperparameters, training loop, batch size, model size, etc.

**What you CANNOT do:**

- Modify `prepare.py`. It is read-only. It contains the fixed evaluation, data loading, tokenizer, and training constants (time budget, sequence length, etc).
- Install new packages or add dependencies. You can only use what's already in `pyproject.toml`.
- Modify the evaluation harness. The `evaluate_bpb` function in `prepare.py` is the ground truth metric.

Source: Karpathy program.md, snapshot 228791f. Original GitHub excerpt: Experimentation.

# What changes are worth testing?

## Goal: lower val_bpb in the same five minutes.

| Change | A trial in train.py | Why try it? |
|---|---|---|
| **Model size** | Try 6 layers instead of 8 | Faster steps, but less capacity |
| **Learning rate** | Try MATRIX_LR = 0.02 | May reduce unstable updates |
| **Batch size** | Use fewer tokens per update | More updates, but noisier gradients |
| **GPU efficiency** | Tune device batch or kernels | Train more tokens per second |

**These are test ideas. A change can also make the score worse.**

Source: Karpathy autoresearch train.py / program.md, snapshot 228791f. Teaching hypotheses, not measured gains.

# Lower val_bpb means better text prediction

## VALIDATION BITS PER BYTE

**How surprised is the model by new text?**

**Better predictions** → **Less surprise**

Source: Karpathy prepare.py; metric intuition

# A completed run returns a score

## Published run: RTX 5070, 12 GB fork

```
val_bpb:          1.225821
peak_vram_mb:     3382.3
```

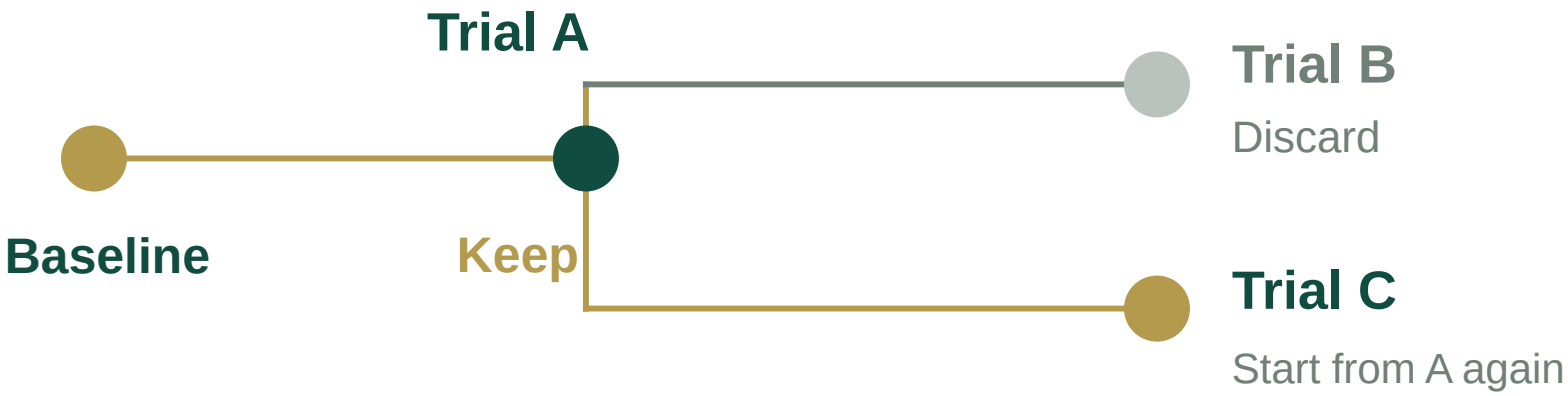
**Text prediction score**

Peak GPU memory

**The agent reads this score to compare its changes.**

Source: fayerman-source, karpathy/autoresearch Discussion #50. Published output, re-typeset.

# Git records the path of code changes




**Every trial stays in the experiment log.**

**The next edit starts from the retained code.**

Source: Karpathy program.md. Teaching sketch of candidate commits and resets.

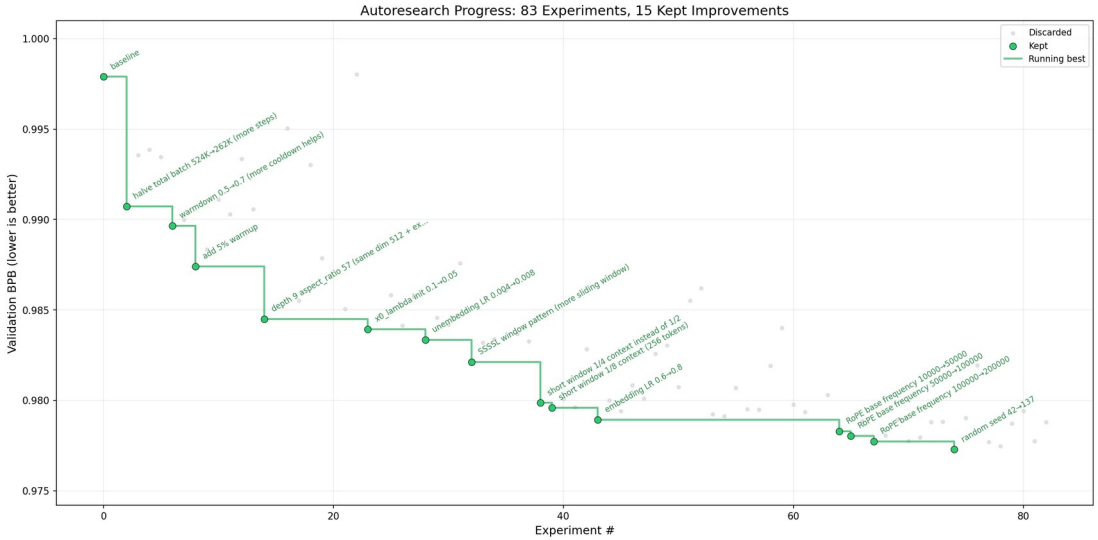
# The agent repeats the experiment loop

```
best = run_baseline()                          # 0  Run the original train.py once
while budget_remains():                        # Stop when the total budget runs out
    trial = agent_edit(best, history)          # 1  Try a different total batch size
    result = run_experiment(trial)             # 2–3  Train, then read val_bpb
    history.record(trial, result)              # Save this edit and its measured result
    best = select(best, trial, result)         # 4  Keep the gain or undo the edit
```

**The experiment history guides the next code change.**

Source: Karpathy program.md. Teaching pseudocode with an explicit total budget.

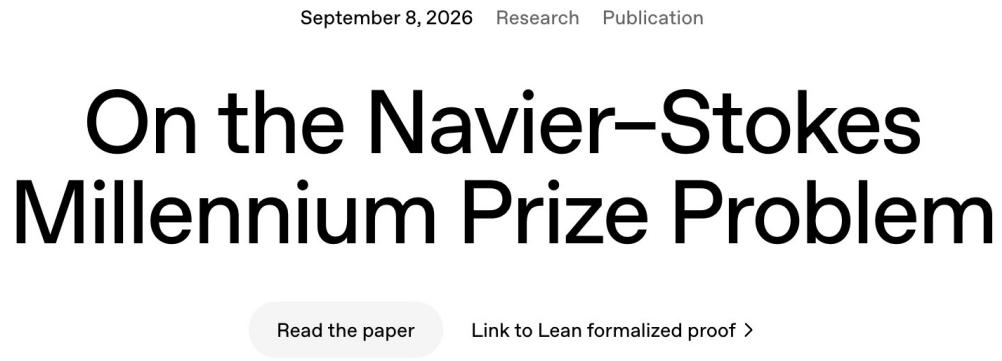
# The published run kept only a few changes




## The 15 kept entries include the starting baseline.

Source: Karpathy, original progress.png (83 experiments, 15 kept entries)

# OpenAI’s mathematical agent swarm




**~10,000**

**concurrent agents**

OpenAI-reported research result

## Different groups explored different approaches.

Researchers helped groups exchange useful findings.

Source: OpenAI, On the Navier–Stokes Millennium Prize Problem, 8 Sep 2026.

# More agents, more search directions

## Repeated idea



## Different directions



**Useful diversity gives the search more chances.**

Source: Anthropic, How we built our multi-agent research system. Concept illustration.

# Some experiments can run together

## Independent trials

| | |
|---|---|
| **GPU 1** | **Smaller batch** |
| GPU 2 | Lower learning rate |
| GPU 3 | Fewer layers |

## Needs their results

**Compare results**
**Choose the next trial**

## Parallel exploration

Three simultaneous runs also need enough compute.

## Then a dependent step

Source: Classroom Autoresearch scenario. Noam Brown interview, 17 Sep 2026.

# A shared research record

## Agent B records an experiment

**FINDINGS.md**

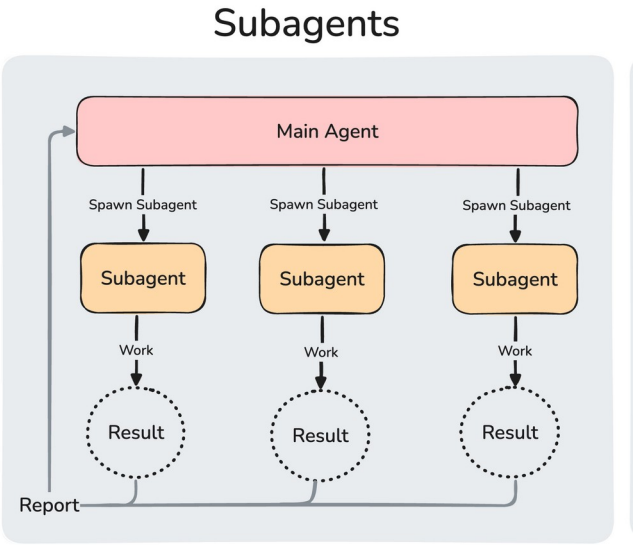
```
Trial B: smaller batch
Result: lower val_bpb in this run
Evidence: trials/B/run.log
Status: needs another seed
```

→ **Agent A reads this before its next trial**

**Other agents load the relevant record into their context.**

Source: Anthropic multi-agent research system. Classroom record below.

# Subagents: delegate a focused task




**A bounded request**

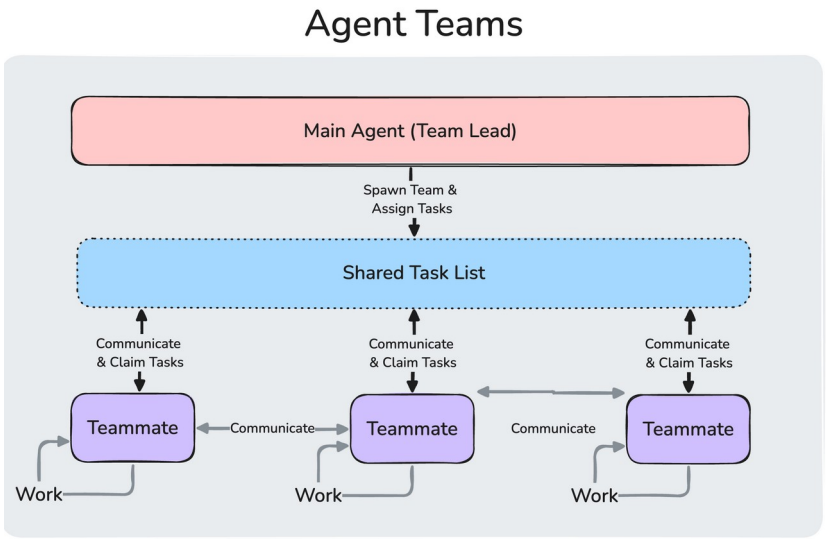
**“Check these training logs.”**

Return findings to the caller.

**The main agent keeps responsibility for the whole task.**

Source: Claude Code documentation, Agent teams / Compare with subagents. Official diagram.

# Agent teams: collaborate on the research




**Own a direction**

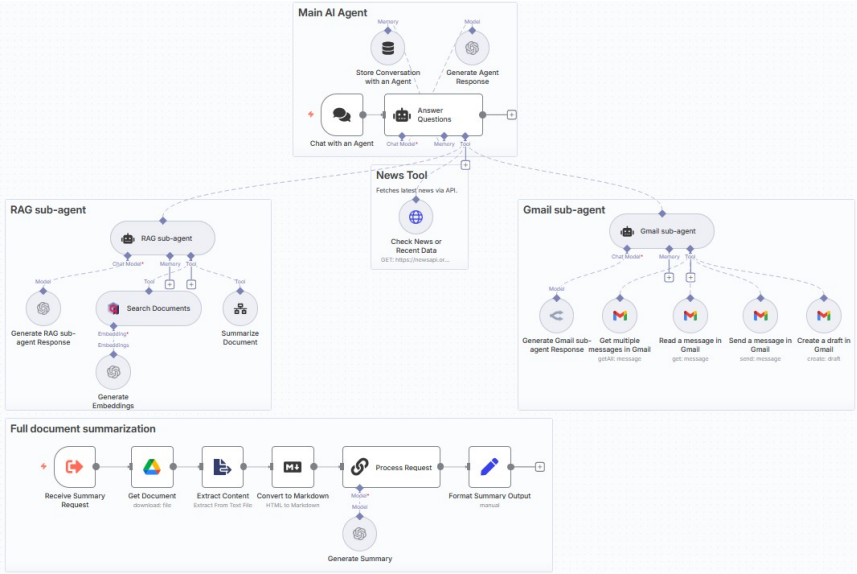
**Share findings**

A lead may still coordinate

**Teammates can discuss and adjust their work.**

Source: Claude Code documentation, Agent teams. Official diagram.

# Workflows: a process you design




**Designed stages**

**Repeatable process**

Agent choices inside each stage

**Useful when the process needs clear checkpoints.**

Source: n8n, Multi-agent systems. Official workflow screenshot.

# Which organization fits the work?

**Subagents**

**“Please check these logs.”**

A focused task returns a result.

**Agent teams**

**“Let us test different hypotheses.”**

Teammates exchange findings.

**Workflows**

**“Every report follows these stages.”**

The application controls the process.

**A system can combine all three patterns.**

Source: Claude Code agent teams documentation / n8n / Anthropic engineering.

# Speed, compute and research quality

| | |
|---|---|
| **Elapsed time** | **How long did we wait?** |
| **Total compute** | **How many GPU-hours and tokens?** |
| **Verified gains** | **Which improvements survived a rerun?** |

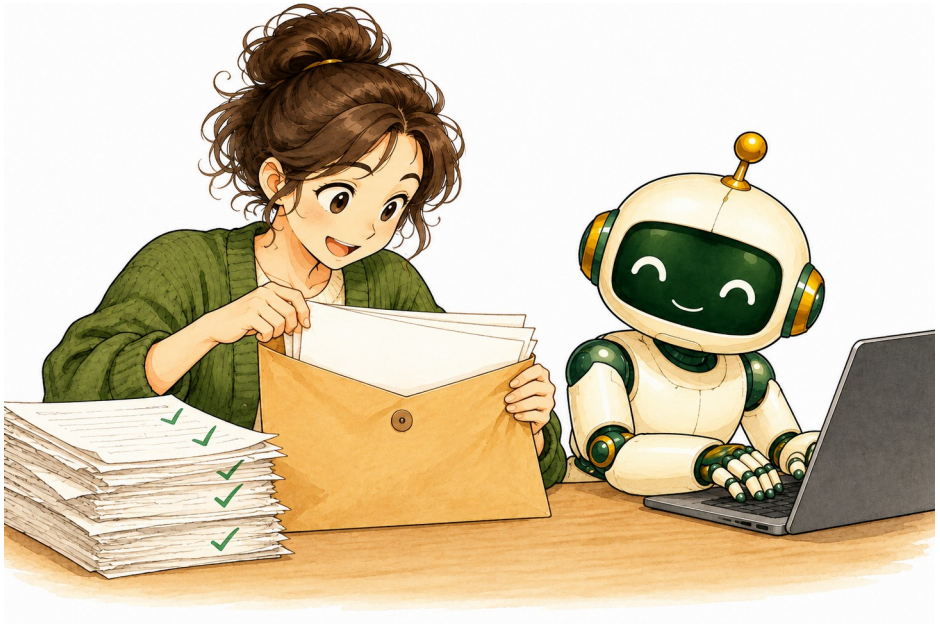
**A faster run may use more resources.**

Source: Noam Brown interview / Anthropic multi-agent research system. Classroom comparison.

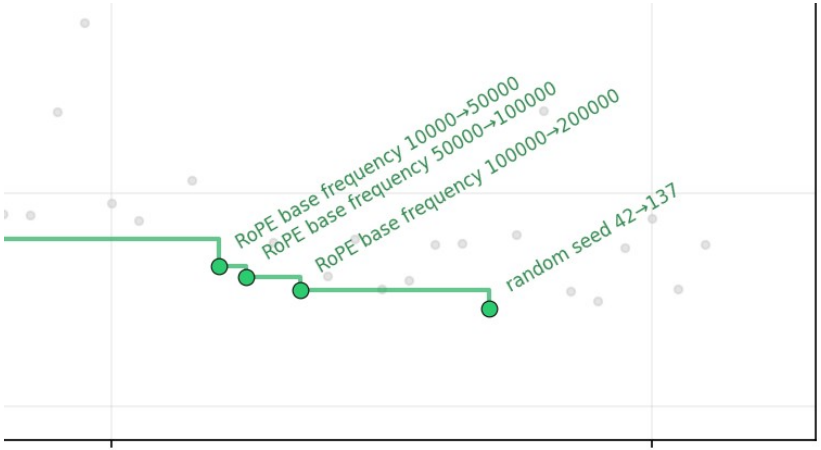
03

# Does the gain hold up?

A lower score still needs a careful check.



# Was the last gain a method change or a lucky seed?




## Same method. Several seeds.

### Repeated runs help show how stable the gain is.

Source: Karpathy progress.png; original annotation crop

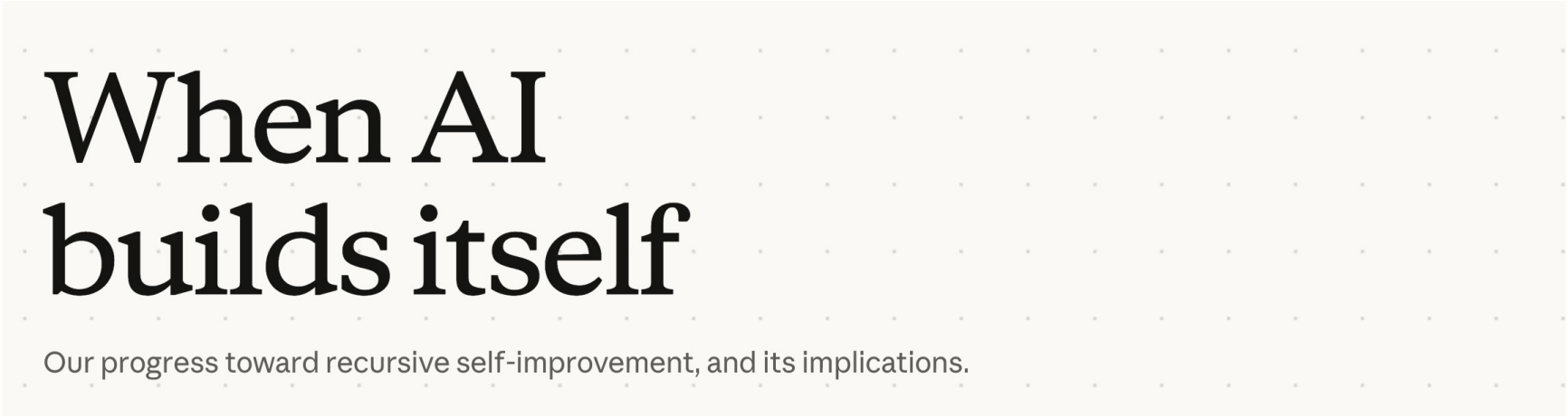
04

# Recursive self-improvement

An improved agent helps build the next improved agent.



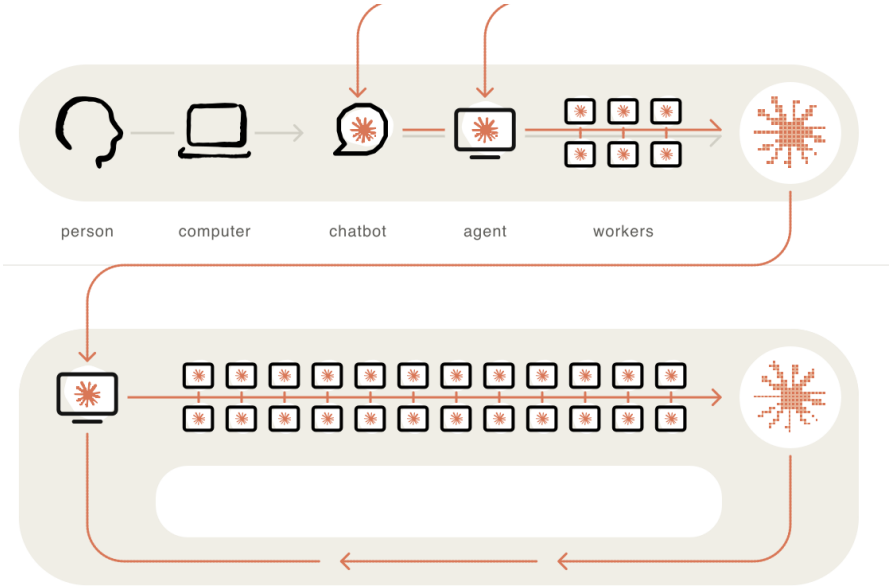
# Anthropic: When AI builds itself




## Could the next AI help build the one after it?

Source: Anthropic Institute, When AI builds itself. Official page screenshot, captured 13 Oct 2026.

# Closing the loop: a possible future




**AI helps people build AI**

**20XX?**

**AI helps build its successor**

Future scenario

Source: Anthropic Institute, When AI builds itself. Original diagram excerpt, final two stages.

# RSI changes who does the next improvement

## AUTORESEARCH EXAMPLE

**Research agent** **improves train.py**

The researcher can stay the same.

## RECURSIVE IMPROVEMENT

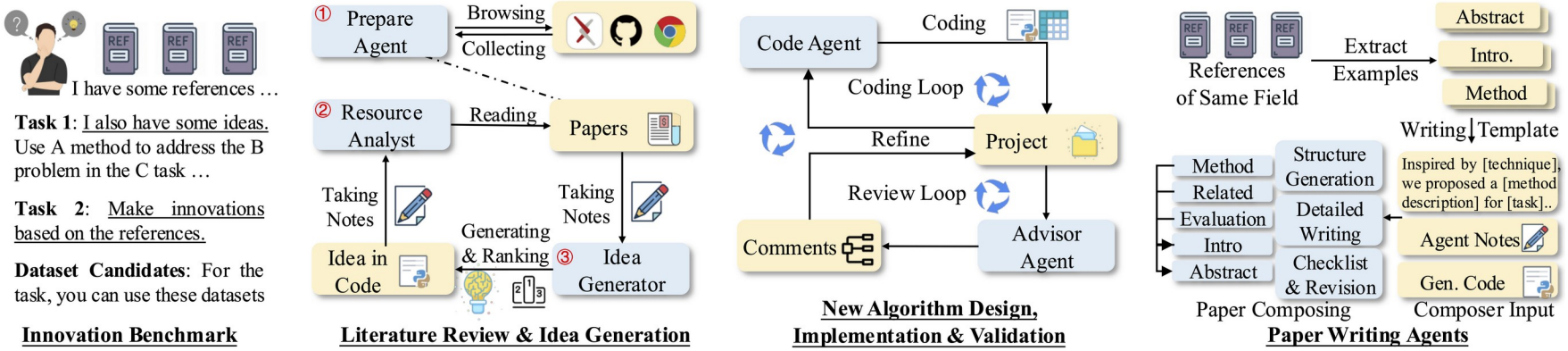
**An improved research method**

**The next round uses that method to improve again.**

Course distinction. Autoresearch source: Karpathy program.md.

# AI-Researcher: the full research workflow

## A research idea can become experiments and a paper.



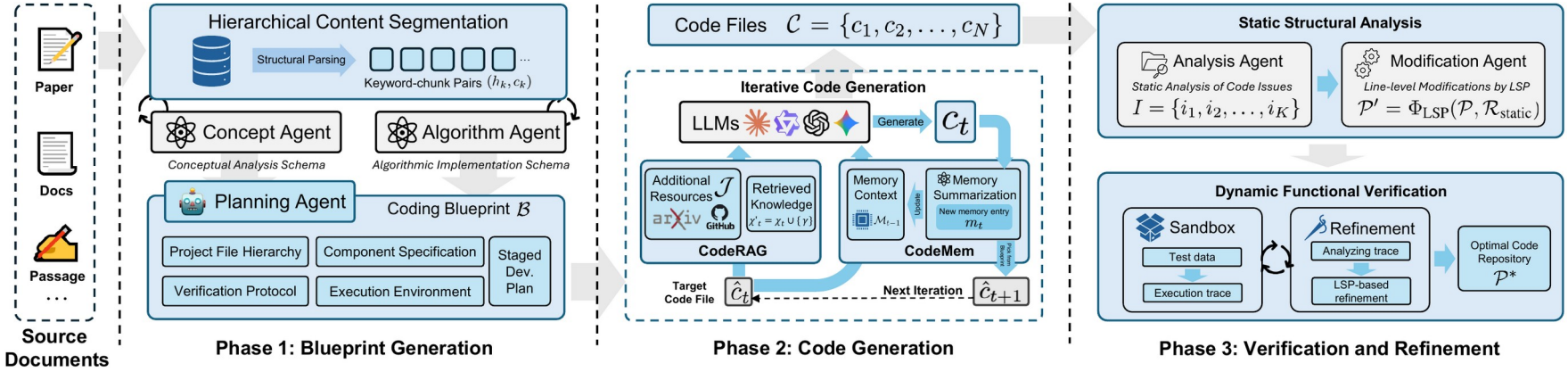

## What improves here? The research output.

Source: HKUDS / AI-Researcher, Tang et al., 2025. Official architecture figure.

# DeepCode: a paper becomes working code

## Research documents

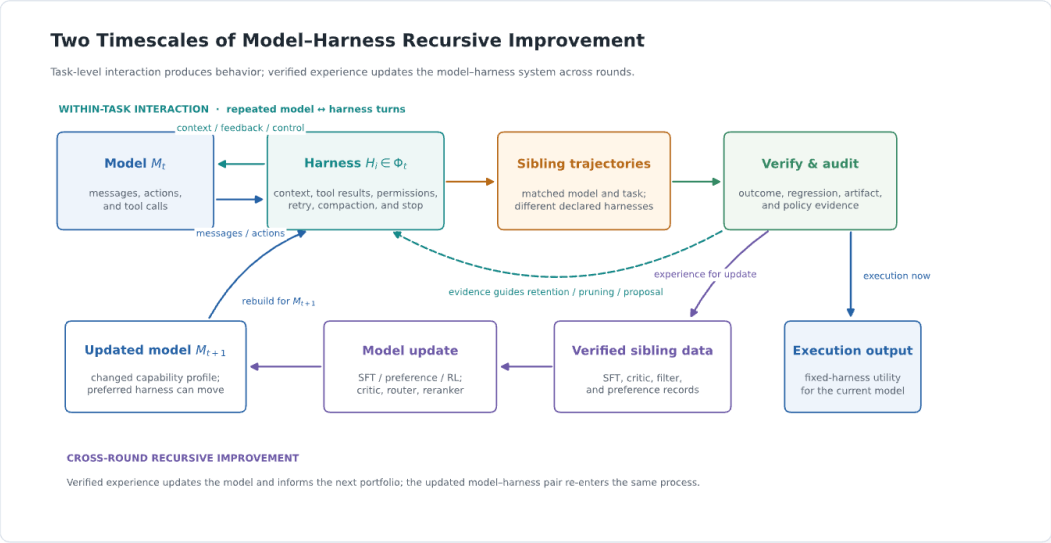
## Implementation and verification




## The feedback improves the generated repository.

Source: HKUDS / DeepCode, Li et al., 2025. Official Paper2Code architecture.

# HELIX: model and harness co-evolution




## Reported evidence

**Harness search with a fixed model**

Model-update data prepared for later

**The reported study does not train an updated model.**

Source: HKUDS / HELIX, Fan and Huang, 2026. Official paper Figure 1 and §9.5.

# Better experiments come before bigger claims

| | |
|---|---|
| **Training** | **Updates model weights** |
| **Autoresearch** | **Tests changes to the training recipe** |
| **RSI** | **Uses an improved researcher again** |

**A clear change, a fair test, and evidence of reuse.**

Course summary: Autoresearch and recursive self-improvement.